DomowyBudżet

DomowyBudżet 1
Dokumentacja techniczna projektu 1
1. Cel i przeznaczenie aplikacji 1
2. Wykorzystane technologie 1
3. Architektura systemu 2
3.1. Schemat architektury 2
3.2. Komunikacja pomiędzy elementami 3
4. Struktura katalogów i plików 3
5. Szczegółowy opis plików 4
5.1. app.py – główny plik aplikacji 4
Importowane biblioteki 4
Inicjalizacja aplikacji 5
Konfiguracja bazy danych 5
Funkcja get_connection() 6
Funkcja calculate_total() 6
Funkcja init_db() 6
Trasa główna – index() 7
Obsługa formularza POST 7
Obsługa formularza GET 8
Obsługa błędów 9
Uruchamianie aplikacji 9
5.2. templates/index.html – interfejs użytkownika 9
Komunikaty Flash 10
Formularz dodawania wydatku 10
Historia wydatków 10
Bootstrap 11
5.3. requirements.txt – zależności Pythona 11
5.4. Dockerfile – budowanie obrazu aplikacji 12
Obraz bazowy 12

Zmienne środowiskowe 12
Katalog roboczy 12
Instalacja zależności 12
Kopiowanie kodu 13
Port i uruchomienie 13
5.5. compose.yaml – zarządzanie kontenerami 13
Usługa web 13
Udostępnienie portu 14
Zmienne środowiskowe 14
Zależność od bazy danych 14
Usługa db 15
Wolumin danych 15
Healthcheck 15
Sieć 16
Wolumin 16
5.6. .env.example – przykładowa konfiguracja 16
5.7. .gitignore – wykluczanie plików z Git 17
5.8. tests/test_app.py – testy automatyczne 18
Test 1 – sumowanie wielu wydatków 18
Test 2 – pusta lista 18
Test 3 – precyzja dziesiętna 18
Test 4 – pojedynczy wydatek 18
5.9. .github/workflows/ci.yml – automatyzacja CI 19
Zadanie test 19
Zadanie docker 19
6. Przepływ działania aplikacji 20
6.1. Uruchomienie środowiska 20
6.2. Wyświetlenie strony głównej 20
6.3. Dodanie nowego wydatku 21
6.4. Aktualizacja historii i sumy 22
6.5. Schemat dodawania wydatku 22
7. Zależności pomiędzy plikami 23

8. Baza danych – struktura i trwałość danych 24
9. Uruchamianie projektu 24
9.1. Wymagania systemowe 24
9.2. Pobranie projektu 24
9.3. Przygotowanie konfiguracji 25
9.4. Walidacja konfiguracji 25
9.5. Uruchomienie aplikacji 25
9.6. Sprawdzenie kontenerów 25
9.7. Dostęp do aplikacji 26
10. Uruchamianie aplikacji na innych komputerach 26
11. Testowanie aplikacji 26
12. CI/CD – GitHub Actions 27
13. Bezpieczeństwo 27
14. Przydatne polecenia administracyjne 28
15. Aktualizacja projektu z GitHub 28
16. Ograniczenia obecnej wersji 28
17. Podsumowanie projektu 29

Dokumentacja techniczna projektu

DomowyBudżet to aplikacja internetowa służąca do zarządzania finansami domowymi. Umożliwia rejestrowanie wydatków, przypisywanie ich do kategorii, przechowywanie danych w relacyjnej bazie MySQL oraz automatyczne obliczanie sumy wszystkich zapisanych wydatków.

Całe środowisko uruchomieniowe jest konteneryzowane za pomocą Docker i Docker Compose. Repozytorium projektu znajduje się na GitHub, a automatyczne testowanie i kontrola jakości kodu realizowane są przez GitHub Actions.

1. Cel i przeznaczenie aplikacji

Głównym celem projektu jest stworzenie prostego systemu umożliwiającego kontrolowanie wydatków gospodarstwa domowego.

Aplikacja pozwala użytkownikowi:

- dodawać nowe wydatki;
- określać kwotę wydatku w PLN;
- przypisywać datę poniesienia wydatku;
- wybierać kategorię wydatku;
- przeglądać historię zapisanych wydatków;
- sprawdzać sumę wszystkich wydatków;
- przechowywać dane w trwałej bazie MySQL.

System automatycznie sprawdza poprawność wprowadzanych danych i zabezpiecza przed zapisaniem nieprawidłowych wartości.

2. Wykorzystane technologie

Technologia	Zastosowanie
Python 3.12	Główny język programowania
Flask	Framework aplikacji internetowej
MySQL 8.4	Relacyjna baza danych
mysql-connector-python	Komunikacja Python–MySQL
HTML5	Struktura interfejsu użytkownika
Bootstrap 5.3	Stylowanie i responsywność
Docker	Konteneryzacja aplikacji
Docker Compose	Zarządzanie kontenerami
pytest	Testowanie automatyczne

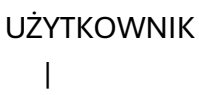
3. Architektura systemu

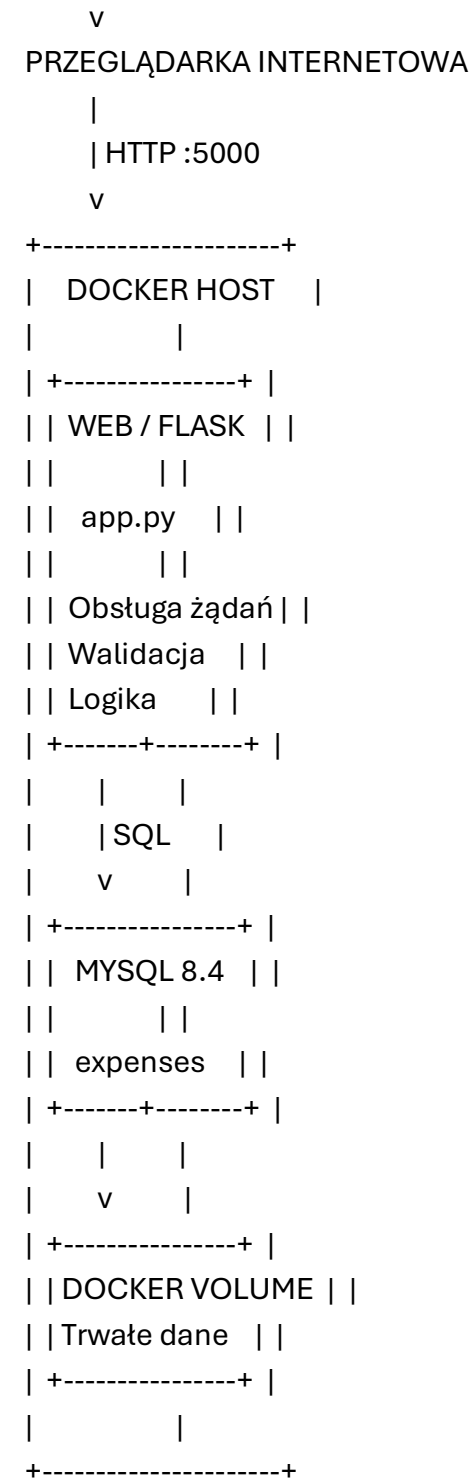
Aplikacja wykorzystuje architekturę klient–serwer składającą się z trzech podstawowych elementów:

1. Przeglądarki internetowej – interfejs użytkownika.
2. Serwera Flask – przetwarzanie danych i logika aplikacji.
3. Bazy MySQL – trwałe przechowywanie wydatków.

Dodatkowo Docker i Docker Compose odpowiadają za uruchomienie i komunikację pomiędzy usługami.

3.1. Schemat architektury





DOCKER COMPOSE

Zarządzanie usługami i siecią

3.2. Komunikacja pomiędzy elementami

Przeglądarka wysyła żądanie HTTP do aplikacji Flask.

Flask odbiera żądanie i wykonuje odpowiednią operację:

- wyświetlenie strony;
- zapis nowego wydatku;
- pobranie historii wydatków;
- obliczenie sumy wydatków.

Jeżeli operacja wymaga dostępu do danych, Flask łączy się z MySQL za pomocą biblioteki mysql-connector-python.

Komunikacja między kontenerami odbywa się przez wewnętrzną sieć Docker.

W konfiguracji aplikacji:

python

#DB_CONFIG = { "host": os.getenv("DB_HOST", "db"), }

Nazwa db jest nazwą usługi MySQL zdefiniowanej w compose.yaml. Docker DNS automatycznie rozpoznaje ją na wewnętrznej sieci.

Baza danych nie musi być udostępniona bezpośrednio na zewnątrz.

4. Struktura katalogów i plików

```
domowy-budzet/
│
├── .github/
│   └── workflows/
│       └── ci.yml
│
├── templates/
│   └── index.html
│
├── tests/
│   └── test_app.py
│
├── app.py
├── Dockerfile
```

```
├── compose.yaml
├── requirements.txt
├── .env.example
├── .gitignore
└── README.md
```

5. Szczegółowy opis plików

5.1. app.py – główny plik aplikacji

Plik app.py jest centralnym elementem całego systemu.

Odpowiada za:

- utworzenie aplikacji Flask;
- konfigurację klucza sesji;
- konfigurację połączenia z MySQL;
- inicjalizację struktury bazy danych;
- obsługę żądań HTTP;
- walidację danych formularza;
- zapisywanie wydatków;
- pobieranie historii wydatków;
- obliczanie sumy wydatków;
- obsługę błędów.

Importowane biblioteki

python

#import os from datetime import date from decimal import Decimal, InvalidOperationimport mysql.connector from flask import Flask, flash, redirect, render_template, request, url_for from mysql.connector import Error

Znaczenie poszczególnych bibliotek:

- os – odczytywanie zmiennych środowiskowych;
- date – obsługa dat;
- Decimal – dokładne operacje na kwotach pieniężnych;
- InvalidOperation – obsługa błędów konwersji wartości dziesiętnych;
- mysql.connector – komunikacja z bazą MySQL;
- Flask – utworzenie aplikacji internetowej;
- request – odbieranie danych z formularza;

- render_template – renderowanie szablonów HTML;
- redirect i url_for – przekierowanie po zapisaniu wydatku;
- flash – wyświetlanie komunikatów użytkownikowi;
- Error – obsługa wyjątków MySQL.

Inicjalizacja aplikacji

python

#app = Flask(__name__) app.secret_key = os.getenv("FLASK_SECRET_KEY", "SECRET-KEY")

Tworzony jest obiekt aplikacji Flask.

secret_key służy do zabezpieczania danych sesyjnych i komunikatów Flask Flash.

W środowisku produkcyjnym należy ustawić własny, losowy klucz przez zmienną FLASK_SECRET_KEY.

Konfiguracja bazy danych

python

#DB_CONFIG = { "host": os.getenv("DB_HOST", "db"), "user": os.getenv("DB_USER", " domowybudzet "), "password": os.getenv("DB_PASSWORD", "domowybudzet, "database": os.getenv("DB_NAME", " domowybudzet "), }

Konfiguracja określa parametry połączenia z MySQL.

Wartości są pobierane ze zmiennych środowiskowych przekazywanych przez Docker Compose.

Dzięki temu dane dostępowe nie muszą być zapisane bezpośrednio w kodzie aplikacji.

Funkcja get_connection()

python

*#def get_connection(): return mysql.connector.connect(**DB_CONFIG)*

Funkcja tworzy połączenie z bazą danych.

Jest wykorzystywana przez pozostałe funkcje aplikacji, które muszą odczytywać lub zapisywać dane.

Funkcja calculate_total()

python

def calculate_total(amounts): return sum((Decimal(str(amount)) for amount in amounts), Decimal("0.00"))

Funkcja oblicza sumę przekazanych kwot.

Wykorzystanie Decimal zamiast float pozwala uniknąć typowych błędów precyzji obliczeń zmiennoprzecinkowych.

Funkcja init_db()

Funkcja odpowiada za przygotowanie struktury bazy danych.

Wykonuje polecenie SQL:

sql

```
#CREATE TABLE IF NOT EXISTS expenses (
  id INT AUTO_INCREMENT PRIMARY KEY,
  amount DECIMAL(10, 2) NOT NULL,
  expense_date DATE NOT NULL,
  category VARCHAR(100) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Znaczenie kolumn:

Kolumna	Typ	Przeznaczenie
id	INT	Unikalny identyfikator wydatku
amount	DECIMAL(10,2)	Kwota wydatku
expense_date	DATE	Data wydatku
category	VARCHAR(100)	Kategoria wydatku
created_at	TIMESTAMP	Data utworzenia rekordu

Trasa główna – index()

python

#@app.route("/", methods=["GET", "POST"]) def index():

Jest to główna trasa aplikacji.

Obsługuje dwa rodzaje żądań HTTP:

- GET – wyświetlenie strony i historii wydatków;
- POST – zapis nowego wydatku.

Obsługa formularza POST

Aplikacja pobiera dane:

python

#amount = Decimal(request.form.get("amount", "")) expense_date = date.fromisoformat(request.form.get("expense_date", "")) category = request.form.get("category", "").strip()

Następnie przeprowadza walidację.

Sprawdzane są:

1. Czy kwota jest prawidłową liczbą.
2. Czy kwota jest większa od zera.
3. Czy kwota nie ma więcej niż dwóch miejsc po przecinku.
4. Czy kwota nie przekracza dopuszczalnego limitu.
5. Czy kategoria nie jest pusta.
6. Czy kategoria nie przekracza 100 znaków.
7. Czy data nie znajduje się w przyszłości.

Po poprawnej walidacji aplikacja wykonuje polecenie SQL:

sql

#INSERT INTO expenses
(amount, expense_date, category)
VALUES (%s, %s, %s);

Dane są przekazywane jako parametry, a nie łączone bezpośrednio z tekstem SQL.

Ogranicza to ryzyko SQL Injection.

Po zapisaniu następuje przekierowanie na stronę główną.

Wykorzystywany jest wzorzec POST/Redirect/GET, który ogranicza ryzyko ponownego wysłania formularza przy odświeżeniu strony.

Obsługa formularza GET

Aplikacja pobiera historię wydatków:

sql

```
SELECT id, amount, expense_date, category
FROM expenses
ORDER BY expense_date DESC, id DESC;
```

Wydatki wyświetlane są od najnowszych.

Następnie obliczana jest suma:

sql

```
#SELECT COALESCE(SUM(amount), 0) AS total
FROM expenses;
```

Pobrane dane są przekazywane do szablonu HTML:

python

```
#render_template( "index.html", expenses=expenses, total=total, today=date.today().isoformat(), )
```

Obsługa błędów

Aplikacja obsługuje nieprawidłowe dane za pomocą wyjątków:

python

```
#except (InvalidOperation, ValueError, TypeError) as error:
```

W przypadku błędów połączenia z MySQL:

python

```
#except Error: app.logger.exception("Błąd połączenia z bazą danych") return "Baza danych jest chwilowo niedostępna.", 503
```

Do logów trafia szczegółowa informacja o błędzie, a użytkownik otrzymuje komunikat o niedostępności bazy.

Uruchamianie aplikacji

Na końcu pliku znajduje się:

python

#if __name__ == "__main__": init_db() app.run(host="0.0.0.0", port=5000)

Najpierw wykonywana jest inicjalizacja tabeli, a następnie uruchamiany jest serwer Flask.

Adres 0.0.0.0 umożliwia nasłuchiwanie na wszystkich interfejsach sieciowych kontenera.

5.2. templates/index.html – interfejs użytkownika

Plik odpowiada za wygląd aplikacji i prezentację danych.

Wykorzystuje:

- HTML5;
- Bootstrap 5.3;
- mechanizm szablonów Jinja2.

Jinja2 jest systemem szablonów zintegrowanym z Flask.

Komunikaty Flash

HTML

Zawijaj wierszeSkopiuj kodPodgląd

{% for category, message in get_flashed_messages(with_categories=true) %}   {{ message }}   {% endfor %}

Odpowiadają za wyświetlanie informacji o poprawnym zapisaniu wydatku lub błędach walidacji.

Formularz dodawania wydatku

Formularz wykorzystuje metodę POST:

HTML

Zawijaj wierszeSkopiuj kodPodgląd

<form method="post" class="row g-3">

Zawiera trzy pola:

- amount – kwota;
- expense_date – data;
- category – kategoria.

Dane są przesyłane do trasy /, obsługiwanej przez funkcję index() w app.py.

Atrybuty HTML, takie jak required, min, max i step, zapewniają podstawową walidację po stronie przeglądarki.

Walidacja serwerowa w app.py pozostaje konieczna, ponieważ ograniczenia HTML można ominąć.

Historia wydatków

HTML

Pętla Jinja2 przechodzi przez wszystkie rekordy przekazane przez Flask.

Wyświetla:

- datę;
- kategorię;
- kwotę.

Jeżeli baza nie zawiera wydatków, wyświetlany jest komunikat:

Brak zapisanych wydatków.

Bootstrap

Framework Bootstrap odpowiada za wygląd interfejsu.

Wykorzystane klasy zapewniają między innymi:

- responsywny układ strony;
- karty;
- formularze;

- przyciski;
- tabelę;
- komunikaty;
- odpowiednie odstępy i typografię.

5.3. requirements.txt – zależności Pythona

Plik zawiera biblioteki wymagane do działania aplikacji i testów.

```
Flask>=3.1,<4
mysql-connector-python>=9.0,<10
pytest>=8.0,<10
```

Opis zależności:

Flask

Framework odpowiedzialny za obsługę aplikacji internetowej, tras HTTP, szablonów i komunikatów.

mysql-connector-python

Sterownik umożliwiający aplikacji Python komunikację z bazą MySQL.

pytest

Framework wykorzystywany do automatycznego testowania funkcji aplikacji.

Plik jest wykorzystywany przez Dockerfile podczas budowania obrazu.

Może być również używany do instalowania zależności w lokalnym środowisku Python.

5.4. Dockerfile – budowanie obrazu aplikacji

Dockerfile zawiera instrukcje potrzebne do przygotowania środowiska uruchomieniowego Flask.

Obraz bazowy

dockerfile

```
FROM python:3.12-slim
```

Wykorzystuje lekki obraz systemu zawierający Python 3.12.

Zmienne środowiskowe

dockerfile

```
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
```

Pierwsza zmienna wyłącza tworzenie plików .pyc.

Druga zapewnia natychmiastowe przekazywanie komunikatów do logów kontenera.

Katalog roboczy

dockerfile

```
WORKDIR /app
```

Ustawia katalog /app jako miejsce pracy wewnątrz kontenera.

Instalacja zależności

dockerfile

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

Docker kopiuje plik zależności do obrazu i instaluje wymagane biblioteki.

Kopiowanie kodu

dockerfile

```
COPY app.py .
COPY templates/ templates/
```

Do obrazu trafia główny plik aplikacji oraz katalog szablonów HTML.

Port i uruchomienie

dockerfile

EXPOSE 5000

CMD ["python", "app.py"]

Port 5000 jest portem aplikacji Flask.

Polecenie CMD określa sposób uruchamiania aplikacji po uruchomieniu kontenera.

Dockerfile jest wykorzystywany przez usługę web w pliku compose.yaml.

5.5. compose.yaml – zarządzanie kontenerami

Docker Compose odpowiada za uruchamianie całego środowiska aplikacji.

Projekt składa się z dwóch usług:

- web – aplikacja Flask;
- db – baza danych MySQL.

Usługa web

yaml

```
web:
  build: .
  container_name: domowy-budzet-web
```

Docker buduje obraz na podstawie Dockerfile znajdującego się w katalogu projektu.

Kontener otrzymuje nazwę domowy-budzet-web.

Udostępnienie portu

yaml

```
ports:
  - "5000:5000"
```

Pierwsza wartość oznacza port komputera hosta, druga port kontenera.

Dzięki temu aplikacja jest dostępna pod adresem:

text

http://localhost:5000

Lub pod zamiast localhost adres pc w sieci, na którym znajdują się pliki

Zmienne środowiskowe

yaml

```
environment:
  DB_HOST: db
  DB_USER: ${DB_USER}
  DB_PASSWORD: ${DB_PASSWORD}
  DB_NAME: ${DB_NAME}
  FLASK_SECRET_KEY: ${FLASK_SECRET_KEY}
```

Docker Compose przekazuje konfigurację do aplikacji Flask.

Zmienne są pobierane z pliku .env.

DB_HOST: db wskazuje nazwę usługi MySQL w sieci Docker.

Zależność od bazy danych

yaml

```
depends_on:
  db:
    condition: service_healthy
```

Aplikacja web jest uruchamiana po uzyskaniu przez bazę statusu healthy.

Nie oznacza to jednak automatycznego ponawiania wszystkich nieudanych połączeń po uruchomieniu. W razie problemów z inicjalizacją należy sprawdzić logi.

Usługa db

yaml

```
db:
  image: mysql:8.4
```

Wykorzystuje oficjalny obraz MySQL w wersji 8.4.

Zmienne środowiskowe:

yaml

```
environment:
  MYSQL_DATABASE: ${DB_NAME}
  MYSQL_USER: ${DB_USER}
  MYSQL_PASSWORD: ${DB_PASSWORD}
  MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
```

Służą do początkowej konfiguracji bazy i kont użytkowników.

Wolumin danych

yaml

```
volumes:
  - domowy_budzet_data:/var/lib/mysql
```

Zapewnia trwałe przechowywanie danych MySQL poza cyklem życia kontenera.

Dzięki temu zatrzymanie lub odtworzenie kontenera nie powoduje automatycznego usunięcia zapisanych wydatków.

Healthcheck

yaml

```
healthcheck:
  test:
    - CMD-SHELL
    - mysqladmin ping -h localhost -u root -p"$${MYSQL_ROOT_PASSWORD}" --silent
  interval: 5s
  timeout: 5s
  retries: 20
  start_period: 20s
```

Docker regularnie sprawdza, czy serwer MySQL odpowiada.

Jeśli kontrola zakończy się poprawnie, kontener otrzymuje status healthy.

Sieć

yaml

```
networks:
  domowy-budzet-network:
    driver: bridge
```

Tworzy wewnętrzną sieć, w której komunikują się kontenery aplikacji i bazy danych.

Wolumin

yaml

```
volumes:
  domowy_budzet_data:
```

Deklaruje logiczny wolumin wykorzystywany przez MySQL.

5.6. .env.example – przykładowa konfiguracja

Plik zawiera przykładowe zmienne środowiskowe potrzebne do uruchomienia aplikacji.

Przykład:

env

```
DB_NAME=domowy_budzet
DB_USER=budzet_user
DB_PASSWORD=change_me
DB_ROOT_PASSWORD=change_root_me
FLASK_SECRET_KEY=change_me_to_a_long_random_secret
```

Plik jest wzorem konfiguracji.

Nie powinien zawierać prawdziwych haseł.

Po pobraniu projektu należy skopiować go do pliku .env:

bash

```
cp .env.example .env
```

Następnie uzupełnia własne dane.

5.7. .gitignore – wykluczanie plików z Git

Plik określa, które pliki i katalogi nie powinny być dodawane do repozytorium.

Przykładowa zawartość:

gitignore

```
.env
venv/
__pycache__/
*.py[cod]
.pytest_cache/
```

Znaczenie:

- .env – lokalne hasła i konfiguracja;
- venv/ – lokalne środowisko Python;
- __pycache__/ – pliki pamięci podręcznej;
- *.py[cod] – pliki skompilowanego Pythona;
- .pytest_cache/ – pamięć podręczna pytest.

Dzięki temu repozytorium zawiera kod źródłowy i potrzebną dokumentację, a nie lokalne dane konfiguracyjne.

5.8. tests/test_app.py – testy automatyczne

Plik zawiera testy funkcji calculate_total() z app.py.

Wykorzystywany jest framework pytest.

Testy sprawdzają poprawność obliczania sumy wydatków.

Test 1 – sumowanie wielu wydatków

python

def test_sum_of_expenses(): amounts = [Decimal("12.50"), Decimal("7.25"), Decimal("10.00"),] assert calculate_total(amounts) == Decimal("29.75")

Sprawdza poprawność sumowania kilku wartości.

Test 2 – pusta lista

python

def test_empty_expenses(): assert calculate_total([]) == Decimal("0.00")

Sprawdza, czy brak wydatków daje zero.

Test 3 – precyzja dziesiętna

python

def test_decimal_precision(): amounts = [Decimal("0.10"), Decimal("0.20")] assert calculate_total(amounts) == Decimal("0.30")

Sprawdza, czy obliczenia pieniężne zachowują dokładność.

Test 4 – pojedynczy wydatek

python

def test_single_expense(): assert calculate_total([Decimal("99.99")]) == Decimal("99.99")

Sprawdza sumowanie jednej wartości.

Testy można uruchomić poleceniem:

bash

python -m pytest -v

Obecny zestaw testów sprawdza funkcję obliczeniową. Nie obejmuje jeszcze testów integracyjnych bazy danych ani testów HTTP.

5.9. .github/workflows/ci.yml – automatyzacja CI

Plik definiuje proces automatycznego sprawdzania projektu w GitHub Actions.

Workflow uruchamia się po:

- wykonaniu push do gałęzi main;
- utworzeniu Pull Request do gałęzi main.

Zadanie test

Pierwszy job odpowiada za kontrolę kodu.

Kolejne kroki:

1. Pobranie kodu przez actions/checkout@v4.
2. Instalacja Pythona 3.12 przez actions/setup-python@v5.
3. Instalacja bibliotek z requirements.txt.
4. Uruchomienie pytest.
5. Kontrola składni przez compileall.

Kontrola składni:

bash

python -m compileall -q app.py tests

Sprawdza poprawność składniową plików Pythona.

Zadanie docker

Drugie zadanie uruchamiane jest dopiero po poprawnym zakończeniu testów.

yaml

needs: test

Następnie budowany jest obraz:

bash

docker build -t domowy-budzet:ci .

Obraz służy do weryfikacji, czy Dockerfile pozwala poprawnie zbudować aplikację.

Obecny workflow buduje obraz, ale nie publikuje go do Docker Hub ani GitHub Container Registry.

6. Przepływ działania aplikacji

Poniżej opisano pełny przebieg operacji od momentu otwarcia strony do zapisania wydatku.

6.1. Uruchomienie środowiska

Użytkownik wykonuje:

bash

docker compose up --build -d

Docker Compose:

1. Odczytuje compose.yaml.
2. Pobiera zmienne z .env.
3. Buduje obraz aplikacji na podstawie Dockerfile.
4. Pobiera obraz MySQL.
5. Tworzy sieć Docker.
6. Tworzy wolumin danych.
7. Uruchamia MySQL.
8. Sprawdza gotowość MySQL przez healthcheck.
9. Uruchamia kontener Flask.

Po uruchomieniu aplikacji wykonywana jest funkcja init_db().

Jeżeli tabela expenses nie istnieje, zostaje utworzona.

6.2. Wyświetlenie strony głównej

Użytkownik otwiera:

text

http://*ip*:5000
gdzie ip to adres pc w sieci na którym znajduje się aplikacja

Przeglądarka wysyła żądanie GET.

Flask odbiera je w funkcji:

python

Plain Text@app.route("/", methods=["GET", "POST"]) def index():

Następnie:

1. Łączy się z MySQL.
2. Pobiera historię wydatków.

3. Oblicza sumę wszystkich kwot.
4. Przekazuje dane do templates/index.html.
5. Jinja2 generuje HTML.
6. Flask odsyła gotową stronę do przeglądarki.

6.3. Dodanie nowego wydatku

Użytkownik uzupełnia formularz.

Przykładowe dane:

Pole	Wartość
Kwota	125.50
Data	2026-11-05
Kategoria	Jedzenie

Po kliknięciu przycisku „Zapisz wydatek” przeglądarka wysyła żądanie POST.

Flask:

1. Odbiera dane formularza.
2. Konwertuje kwotę na Decimal.
3. Konwertuje datę na obiekt date.
4. Pobiera kategorię.
5. Sprawdza poprawność danych.
6. Łączy się z MySQL.
7. Wykonuje polecenie INSERT.
8. Zatwierdza transakcję przez commit().
9. Wyświetla komunikat o sukcesie.
10. Przekierowuje użytkownika na stronę główną.

6.4. Aktualizacja historii i sumy

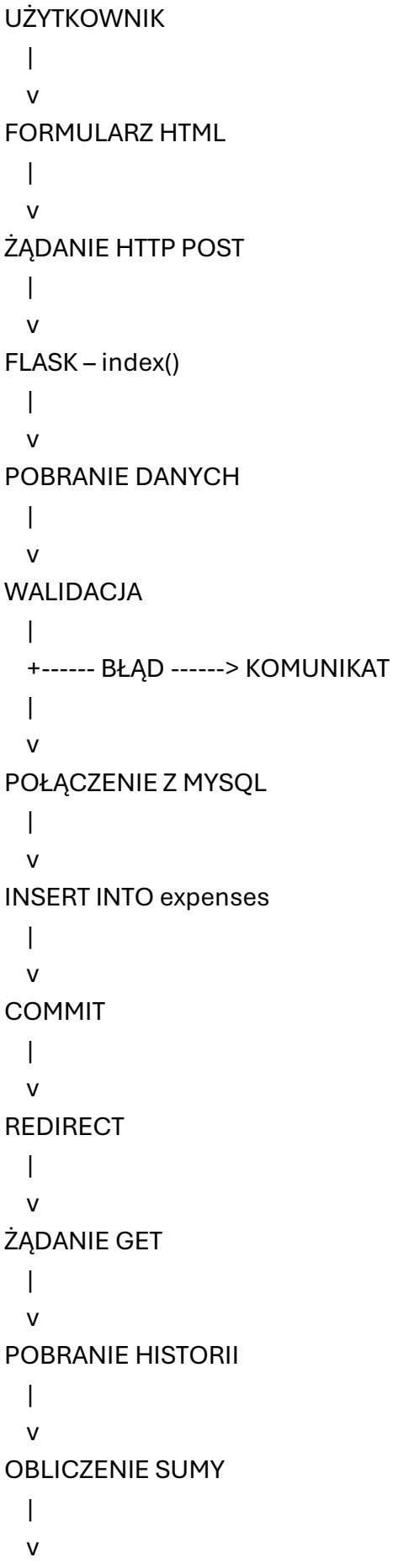
Po zapisaniu wydatku przeglądarka wykonuje ponowne żądanie GET.

Flask pobiera aktualną historię wydatków i ponownie oblicza sumę.

Dzięki temu nowy wydatek pojawia się w tabeli, a całkowita kwota zostaje automatycznie zaktualizowana.

6.5. Schemat dodawania wydatku

text


UŻYTKOWNIK
|
v
FORMULARZ HTML
|
v
ŻĄDANIE HTTP POST
|
v
FLASK – index()
|
v
POBRANIE DANYCH
|
v
WALIDACJA
|
+------ BŁĄD ------> KOMUNIKAT
|
v
POŁĄCZENIE Z MYSQL
|
v
INSERT INTO expenses
|
v
COMMIT
|
v
REDIRECT
|
v
ŻĄDANIE GET
|
v
POBRANIE HISTORII
|
v
OBLICZENIE SUMY
|
v

RENDEROWANIE HTML
  |
  v
ZAKTUALIZOWANA STRONA

7. Zależności pomiędzy plikami

Najważniejsze powiązania w projekcie:

Plik źródłowy	Plik lub element docelowy	Zależność
app.py	requirements.txt	Wykorzystuje zainstalowane biblioteki
app.py	.env	Pobiera konfigurację i dane dostępowe
app.py	MySQL	Zapisuje i odczytuje wydatki
app.py	templates/index.html	Przekazuje dane do interfejsu
Dockerfile	requirements.txt	Instaluje zależności
Dockerfile	app.py	Kopiuje kod aplikacji
Dockerfile	templates/	Kopiuje szablony HTML
compose.yaml	Dockerfile	Buduje kontener web
compose.yaml	.env	Przekazuje zmienne środowiskowe
compose.yaml	MySQL	Uruchamia bazę danych
compose.yaml	Docker Volume	Przechowuje dane
templates/index.html	app.py	Odbiera dane przekazane przez Flask
tests/test_app.py	app.py	Testuje funkcję obliczającą sumę
ci.yml	requirements.txt	Instaluje biblioteki na potrzeby CI
ci.yml	tests/test_app.py	Uruchamia testy
ci.yml	Dockerfile	Buduje obraz aplikacji
README.md	Cały projekt	Dokumentuje działanie i konfigurację

8. Baza danych – struktura i trwałość danych

Aplikacja korzysta z jednej tabeli expenses.

Relacja pomiędzy aplikacją i bazą danych jest realizowana przez sterownik MySQL Connector.

Każdy wydatek jest przechowywany jako osobny rekord.

Dane nie są przechowywane w pamięci aplikacji Flask, lecz w MySQL.

Docker Volume zapewnia ich trwałość po zatrzymaniu i ponownym uruchomieniu kontenerów.

9. Uruchamianie projektu

9.1. Wymagania systemowe

Do uruchomienia aplikacji potrzebne są:

- Linux, Windows lub macOS;
- Docker Engine lub Docker Desktop;
- Docker Compose;
- Git;
- dostęp do repozytorium GitHub.

9.2. Pobranie projektu

bash

```
git clone https://github.com/zslitstudent07/domowy-budzet.git
cd domowy-budzet
```

9.3. Przygotowanie konfiguracji

bash

```
cp .env.example .env
```

Uzupełnij dane dostępowe:

env

```
DB_NAME=domowy_budzet
DB_USER=budzet_user
DB_PASSWORD=TwojeSilneHaslo
DB_ROOT_PASSWORD=InneSilneHaslo
FLASK_SECRET_KEY=dlugi_losowy_klucz
```

9.4. Walidacja konfiguracji

bash

```
docker compose config
```

9.5. Uruchomienie aplikacji

bash

```
docker compose up --build -d
```

9.6. Sprawdzenie kontenerów

bash

```
docker compose ps
```

Oczekiwane usługi:

- domowy-budzet-web;
- domowy-budzet-db.

Kontener MySQL powinien mieć status healthy.

9.7. Dostęp do aplikacji

http://adres_ip:5000

adres_ip – ip pc na którym znajdują się pliki

10. Uruchamianie aplikacji na innych komputerach

Projekt można pobrać i uruchomić na innym komputerze posiadającym Docker.

Każdy komputer powinien posiadać własny plik .env.

Domyślnie każdy komputer uruchamia własny kontener MySQL i własny wolumin danych.

Oznacza to, że dane zapisane na jednym komputerze nie są automatycznie synchronizowane z drugim.

Jeżeli aplikacja ma być dostępna dla wielu użytkowników ze wspólną bazą, należy uruchomić ją na jednym serwerze i zapewnić dostęp do niego przez sieć.

11. Testowanie aplikacji

Testy jednostkowe uruchamiane są przez pytest.

Instalacja środowiska lokalnego:

bash

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Uruchomienie:

bash

```
python -m pytest -v
```

Testy obejmują:

- poprawność sumowania wydatków;
- poprawne zachowanie dla pustej listy;
- precyzję obliczeń dziesiętnych;
- obsługę pojedynczego wydatku.

Dodatkowo kontrola składni:

bash

```
python -m compileall -q app.py tests
```

12. CI/CD – GitHub Actions

Po przesłaniu zmian do gałęzi main lub utworzeniu Pull Request uruchamia się workflow.

Proces obejmuje:

1. Pobranie kodu.
2. Przygotowanie Pythona 3.12.
3. Instalację zależności.
4. Uruchomienie testów.
5. Sprawdzenie składni.
6. Budowanie obrazu Docker.

Dzięki temu podstawowe błędy mogą zostać wykryte jeszcze przed wdrożeniem zmian.

13. Bezpieczeństwo

W projekcie zastosowano kilka podstawowych mechanizmów bezpieczeństwa:

- dane dostępowe do MySQL są przechowywane w zmiennych środowiskowych;
- plik .env jest wykluczony z repozytorium;
- zapytania SQL wykorzystują parametry;
- dane formularza są walidowane po stronie serwera;
- kwoty pieniężne obsługiwane są przez Decimal;
- komunikacja między kontenerami odbywa się przez wydzieloną sieć Docker;
- dane MySQL są przechowywane w woluminie.

Aplikacja nie zawiera obecnie systemu logowania, rejestracji użytkowników ani rozdzielania wydatków pomiędzy użytkowników.

14. Przydatne polecenia administracyjne

Operacja	Polecenie
Uruchomienie	docker compose up -d
Budowanie i uruchomienie	docker compose up --build -d
Zatrzymanie	docker compose stop
Wznowienie	docker compose start
Restart	docker compose restart
Status kontenerów	docker compose ps
Logi aplikacji	docker compose logs -f web
Logi MySQL	docker compose logs -f db
Zatrzymanie i usunięcie kontenerów	docker compose down
Sprawdzenie konfiguracji	docker compose config

Uwaga: docker compose down -v usuwa również woluminy zadeklarowane w projekcie. Nie należy używać tego polecenia bez wcześniejszego zabezpieczenia danych.

15. Aktualizacja projektu z GitHub

Aby pobrać najnowsze zmiany:

bash

```
cd ~/domowy-budzet
git pull origin main
```

Następnie przebudować aplikację:

bash

```
docker compose up --build -d
```

Dane bazy pozostają w woluminie, o ile nie zmieniono jego nazwy ani nie usunięto go.

16. Ograniczenia obecnej wersji

Aktualna wersja DomowyBudżet koncentruje się na podstawowej ewidencji wydatków.

Nie zawiera jeszcze:

- rejestracji i logowania użytkowników;
- edycji istniejących wydatków;
- usuwania wydatków;
- filtrowania według dat;
- raportów miesięcznych;
- wykresów wydatków;
- eksportu danych do CSV lub PDF;
- budżetów przypisanych do kategorii;
- synchronizacji danych pomiędzy niezależnymi instalacjami.

Funkcjonalności te mogą zostać dodane w kolejnych wersjach projektu.

17. Podsumowanie projektu

DomowyBudżet jest aplikacją webową wykorzystującą współpracę kilku technologii.

Python i Flask odpowiadają za logikę biznesową i obsługę żądań.

HTML, Jinja2 i Bootstrap odpowiadają za prezentację danych użytkownikowi.

MySQL zapewnia trwałe przechowywanie informacji o wydatkach.

Docker i Docker Compose umożliwiają uruchomienie całego środowiska w kontenerach.

pytest sprawdza poprawność najważniejszej funkcji obliczeniowej.

Poszczególne elementy tworzą jeden spójny system, w którym każda warstwa ma określoną odpowiedzialność.

Nazwa projektu: DomowyBudżet

Repozytorium: domowy-budzet

Typ aplikacji: aplikacja internetowa

Architektura: klient–serwer

Baza danych: MySQL 8.4

Środowisko uruchomieniowe: Docker / Docker Compose

Język programowania: Python 3.12